Assignment 3: подготовка фестиваля

Для фестиваля нужно соединить площадки, составить план подготовки и организовать доставку. Реализуйте набор алгоритмов для трёх моделей: неориентированные кабельные сети, ориентированные зависимости и ориентированные маршруты доставки.

Дедлайн: 18 октября 2026 года, 23:59 (Asia/Qyzylorda, UTC+5). Работайте в своей копии шаблона GitHub. В Moodle сдайте ZIP-снимок коммита по инструкции в конце.

Цели и предварительные знания

Реализуйте и объясните DSU, Краскала, Прима, SCC Косарайю, топологическую сортировку через DFS и кратчайшие пути в DAG. Научитесь проверять восстановленные ответы, учитывать равные оптимумы и граничные случаи, восстанавливать маршруты и обосновывать сложность. Необходимы списки смежности, DFS, сортировка, очередь с приоритетом и 64-битная арифметика.

Интерфейс программы

На каждую строку входа с одним JSON-объектом выведите одну строку с JSON-результатом. Один запрос содержит одну задачу. Не выводите приглашения и отладочные сообщения в stdout. Перед каждым запросом сбрасывайте состояние. Готовые адаптеры отвечают за ввод и вывод. Вход соответствует опубликованным ограничениям; некорректный JSON не оценивается. Кодировка — UTF-8, регистр имён задач важен.

Вершины и рёбра нумеруются с нуля. ID ребра — его позиция во входном массиве. Стоимости, расстояния, веса и счётчики должны поддерживать знаковые 64-битные целые числа. null обозначает недостижимую вершину.

Задача 1. Связность — DSU

Соединяйте площадки по одной связи. После каждой операции верните число компонент и размер наибольшей.

Вход: {task:dsu,n,operations:[[u,v],...]}

Выход: {states:[[components,largest],...]}

Ограничения: $0 \le n \le 100000$; $0 \le q \le 250000$ операций. При n=0 операций нет. Повторное объединение и объединение вершины с собой допустимы.

Реализуйте find и union со сжатием путей и объединением по размеру или рангу. Изначально есть n одноэлементных компонент; при n=0 largest=0. Записывайте состояние после каждой операции. Объединение внутри одной компоненты ничего не меняет.

Сложность: O((n+q) alpha(n)) амортизированного времени; O(n+q) памяти с учётом результата.

Пример входа:

```
{"task":"dsu","n":4,"operations":[[0,1],[1,2],[0,2],[3,3]]}
```

Допустимый результат:

```
{"states":[[3,2],[2,3],[2,3],[2,3]]}
```

Задача 2. Кабельная сеть — Краскал

Постройте минимальный остовный лес неориентированного взвешенного графа.

Вход: {task:kruskal,n,edges:[[u,v,weight],...]}

Выход: {cost,edge_ids:[...],components}

Ограничения: $0 \le n \le 100000$; $0 \le m \le 250000$; $-1000000000 \le weight \le 1000000000$. Петли и параллельные рёбра допустимы.

Реализуйте Краскала с DSU. Учитывайте все компоненты, включая изолированные вершины; при n=0 components=0. cost — сумма весов леса. edge_ids содержит ID исходных рёбер без повторений. Петли не включаются. Принимается любой оптимальный лес. Отрицательные веса допустимы.

Сложность: O(n+m log(m+1)) времени; O(n+m) памяти.

Пример входа:

```
{"task":"kruskal","n":4,"edges":[[0,1,4],[1,2,2],[0,2,3]]}
```

Допустимый результат:

```
{"cost":5,"edge_ids":[1,2],"components":2}
```

Вершина 3 изолирована: лес стоит 5 и не соединяет все площадки в одну компоненту.

Задача 2. Кабельная сеть — Прим

Независимо реализуйте построение того же минимального остовного леса, расширяя по одной компоненте.

Вход: {task:prim,n,edges:[[u,v,weight],...]}

Выход: {cost,edge_ids:[...],components}

Ограничения и требования к лесу совпадают с задачей Краскала. Используйте списки смежности и очередь с приоритетом. Начинайте заново в каждой ещё не посещённой компоненте. Стоимости должны совпадать с Краскалом; наборы ID могут отличаться. Сравните реализации на трёх размерах входа в ANALYSIS.md.

Сложность: O(n+m log(m+1)) с ленивой кучей; O((n+m) log(n+1)) с индексированной кучей; O(n+m) памяти.

Пример входа:

```
{"task":"prim","n":3,"edges":[[0,1,1],[1,2,1],[0,2,1]]}
```

Допустимый результат:

```
{"cost":2,"edge_ids":[0,2],"components":1}
```

Оптимальных ответов несколько: проверяется корректность леса, а не совпадение с одним списком рёбер.

Задача 3. Циклические зависимости — SCC

Найдите компоненты сильной связности ориентированного графа и постройте конденсацию.

Вход: {task:scc,n,edges:[[u,v],...]}

Выход: {component:[метка каждой вершины],condensation:[[from_label,to_label],...]}

Ограничения: $0 \le n \le 100000$; $0 \le m \le 250000$. Повторяющиеся рёбра и петли допустимы.

Реализуйте Косарайю: два прохода DFS и транспонированный граф. Метки — неотрицательные целые числа; значения и порядок произвольны. Одинаковая метка должна означать ровно одну SCC. Конденсация содержит каждую пару различных компонент, связанную исходным ребром, ровно один раз и не содержит внутренних рёбер. Учтите изолированные вершины и глубокие графы. При n=0 оба массива пусты. Конденсация является DAG, но циклы внутри SCC остаются проблемой исходного расписания.

Сложность: O(n+m) ожидаемого времени при устранении повторов хеш-множеством; O(n+m) памяти.

Пример входа:

```
{"task":"scc","n":4,"edges":[[0,1],[1,0],[1,2],[0,2],[2,3]]}
```

Допустимый результат:

```
{"component":[5,5,7,9],"condensation":[[5,7],[7,9]]}
```

Вершины 0 и 1 принадлежат одной SCC; две связи между компонентами превращаются в одну.

Задача 4. План подготовки — топологический порядок

Вход: {task:topo,n,edges:[[u,v],...]}

Выход: {acyclic:boolean,order:[...]}

Ограничения совпадают с ориентированным графом в SCC. Ребро u→v означает, что u должна завершиться до начала v.

Используйте DFS с состояниями «не посещена / активна / завершена» и обратным порядком завершения. Ребро к активной вершине означает цикл. При цикле в любой части графа верните acyclic=false и order=[]. Для DAG перечислите каждую вершину ровно один раз с соблюдением всех рёбер. Принимается любой допустимый порядок. Пустой граф: acyclic=true, order=[].

Сложность: O(n+m) времени и памяти, включая явный стек DFS.

Пример входа:

```
{"task":"topo","n":4,"edges":[[0,2],[1,2],[2,3]]}
```

Допустимый результат:

```
{"acyclic":true,"order":[1,0,2,3]}
```

Вершины 0 и 1 можно переставить; обе должны предшествовать 2.

Задача 5. Доставка — кратчайшие пути в DAG

Вход: {task:dag,n,edges:[[u,v,weight],...],source}

Выход: {acyclic:boolean,distance:[целое число или null для каждой вершины],parent_edge:[ID ребра или −1 для каждой вершины]}

Ограничения: $1 \le n \le 100000$; $0 \le m \le 250000$; $-1000000000 \le weight \le 1000000000$; $0 \le source < n$. Граф ориентированный; параллельные рёбра и петли допустимы на входе, но циклический граф нужно отклонить.

Проверьте ацикличность всего графа, затем релаксируйте рёбра в топологическом порядке. При любом цикле, даже недостижимом из source, верните acyclic=false и два пустых массива. Иначе у источника distance=0 и parent_edge=−1; у недостижимых вершин distance=null и parent_edge=−1. Для каждой другой вершины родительское ребро должно заканчиваться в ней и восстанавливать оптимальный путь из источника. Напишите вспомогательную функцию восстановления маршрута и покажите пример её использования. Не релаксируйте из бесконечности. Любой кратчайший маршрут допустим; остовное дерево решает другую задачу.

Сложность: O(n+m) времени и памяти. Восстановление пути — O(длины пути).

Пример входа:

```
{"task":"dag","n":5,"source":0,"edges":[[0,1,3],[0,2,8],[1,2,-7],[2,3,4]]}
```

Допустимый результат:

```
{"acyclic":true,"distance":[0,3,-4,0,null],"parent_edge":[-1,0,2,3,-1]}
```

Маршрут к 3: 0→1→2→3, стоимость 0. Вершина 4 недостижима. Отрицательные веса в DAG допустимы.

Анализ и собственные тесты

Заполните ANALYSIS.md на английском или русском: по 3–5 предложений на алгоритм об инварианте, шагах, граничных случаях и сложности. Объясните сброс состояния между запросами, обработку глубокого DFS, восстановление леса, конденсации и пути. Сравните Краскала и Прима на одних графах трёх воспроизводимых размеров. Укажите n, m, seed или файлы входа, версии среды, команды, число повторов, медианное время и выводы. Время используется для анализа, а не рейтинга скорости.

Добавьте свои входы, ожидаемые и фактические результаты в tests/student/. Проверьте обычные случаи и подходящие границы: пустые и несвязные графы, петли, параллельные рёбра, равные оптимумы, отрицательные веса, циклы, недостижимые вершины и повторные запросы. Это дополнение к публичным тестам. По действующей политике силлабуса укажите помощь и источники, их вклад и собственную проверку. Вы должны понимать реализацию; дополнительных ограничений на ИИ нет. По силлабусу ИИ

разрешён для обсуждения идей, отладки и предложений кода с письменным раскрытием использования; студент отвечает за корректность, оригинальность и качество. Работа, сгенерированная ИИ и сданная без изменений или указания помощи, рассматривается как плагиат. Преподаватель вправе проверить понимание сданной работы.

Практическая часть: 50 баллов

Алгоритм	Основное	Границы	Восстановление	Масштаб	Всего
DSU	4	2	0	2	8
Краскал	4	0	2	1	7
Прим	4	0	2	1	7
SCC	5	0	3	2	10
Топологический порядок	4	2	0	2	8
Пути в DAG	5	0	3	2	10
Итого					50

Основные проверки оценивают значения и базовое поведение; границы — опубликованные граничные случаи; восстановление — корректные оптимальные леса, пути и точную конденсацию; масштаб — корректные ответы на больших входах. В каждой группе: баллы группы × пройденные проверки / все проверки, с округлением half-up до 0,01. Сумма не больше 50. Верные основные значения могут получить баллы при неверном восстановлении. Ошибка сборки не позволяет пройти проверки; тайм-аут теряет затронутые проверки. Ошибка проверяющей системы не даёт оценку до исправления и повторного запуска. Дополнительные тесты не меняют максимум группы и не вводят неопубликованные требования.

Официально проверяется ZIP из Moodle фиксированной версией преподавательской системы, а не workflow студента. По тестам нельзя доказать выбор алгоритма: реализуйте указанные методы и оставьте код понятным для просмотра. Анализ помогает проверке; отдельной скрытой группы баллов за него нет. Разрешены стандартные коллекции, сортировки и очереди с приоритетом; внешняя библиотека алгоритмов не заменяет требуемую реализацию.

Сборка: 60 секунд. Пакет проверок каждой группы: 20 секунд, один CPU, 768 MiB памяти контейнера (Java heap до 512 MiB), 128 процессов, до 8 MiB вывода UTF-8. Типичные большие проверки: до 50000 вершин, 100000 операций DSU и 60000 рёбер MST в пределах полных ограничений выше. C++ проверяется на Linux GCC.

Отдельный квиз на индивидуальное понимание — 50 баллов; условия объявят позже. Существующие тренировочные PDF необязательны и не оцениваются. Практика и квиз дают оценку за задание из 100. Веса силлабуса сохраняются.

От GitHub до Moodle

1. Откройте https://github.com/RZhaniya/daa-assignment-3-festival и выберите Use this template → Create a new repository. Можно использовать публичную или приватную копию; при необходимости предоставьте преподавателю доступ к приватной. Установите Git, Python 3.12+ и среду: JDK 17+, компилятор C++17 или .NET SDK 8. Клонируйте свою копию через Git, а не скачивайте ZIP.

2. Выберите Java, Python, Cpp или CSharp в submission.json. Реализуйте алгоритмы в source/ выбранного языка; дополнительные исходники размещайте непосредственно в этой папке. Сохраните JSON-адаптеры.

3. Выполните команды ниже из папки репозитория. Setup проверяет обмен данными; TODO ожидаемо не проходит --check. В Windows можно использовать py -3 вместо python. Язык берётся из submission.json; --language меняет его для одного запуска.

```
python tools/run.py --setup
python tools/run.py --check
python tools/run.py --input examples/dsu.jsonl
python tools/check_submission.py
```

4. Заполните ANALYSIS.md, SUBMISSION.md и собственные тесты. Изучите локальные результаты и GitHub Actions → Public checks после push. Если предложено, включите Actions в своей копии. Публичные проверки помогают подготовиться, но не гарантируют полный балл или авторство.

5. Сделайте commit и push. Убедитесь, что git status не показывает изменений. Создайте пакет, подставив числовой ID студента:

```
python tools/package_submission.py --student-id YOUR_NUMERIC_STUDENT_ID
```

6. В Assignment 3 загрузите .submission/Assignment3_<ID>.zip. В Online text вставьте три строки из .submission/Moodle_submission.txt: ссылку на репозиторий, полный SHA коммита из 40 символов и язык. Сохраните, нажмите Submit assignment и проверьте статус Submitted for grading до дедлайна. Черновик не считается завершённой сдачей.

Пакет содержит исходники выбранного языка, manifest, анализ, сведения о сдаче, README и свои тесты. Сборки, остальные языки, публичные инструменты проверки и тренировочные PDF не включаются. Упаковка фиксирует снимок, но не готовность всех требований. Сохраните ZIP и подтверждение Moodle. Чтобы исправить до дедлайна, сделайте новый commit и push, создайте пакет заново и замените сдачу через доступные элементы Moodle. Если работа заблокирована, обратитесь к преподавателю до дедлайна. Поздние изменения GitHub не обновляют ZIP. Своевременность определяется записью Moodle; одобренное индивидуальное продление имеет приоритет.